

Auto-Citation Proof of Concept

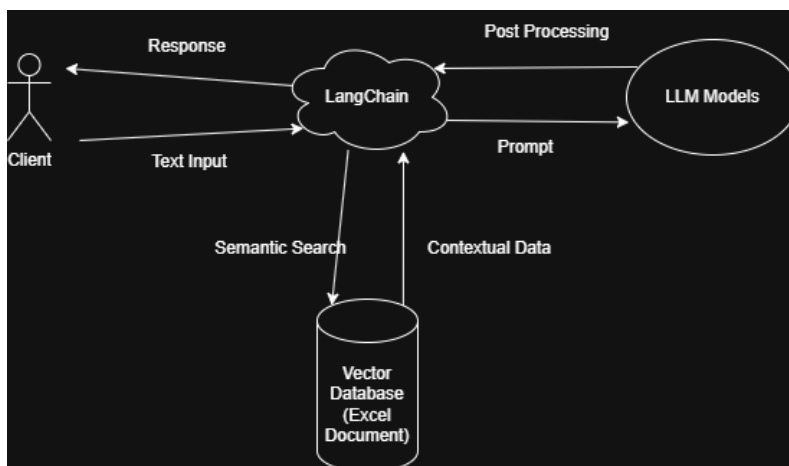
- i. Introduction
- ii. Design Proposal
- iii. Success Criteria
- iv. PoC Results

1.Introduction

Goal of the project will be release of a program that cites references of a written statement automatically via LLM models.

2.Design Proposal

2.1 RAG Architecture



In this project I'll use RAG Architecture for given task. For this PoC part I will use LangChain as framework. Vector database will be an excel document that will contain 10-15 random statement that will use as ground truth. LLM models that will used on this project will be listed.

LLM Models for this project:

- Llama 3 (8B)
- Mistral (7B)
- Phi-3 Medium Mini (3.8B)
- Qwen 3 (8B)

Articles That Will Proposed to Project:

Atomic Claim Extraction:

- *Fact in fragments: Deconstructing complex claims via LLM-based atomic fact extraction and verification* (Liwen Zheng, Chaozhuo Li, Zheng Liu, Feiran Huang, Haoran Jia, Zaisheng Ye, Xi Zhang)

- *Claim Extraction for Fact-Checking: Data, Models, and Automated Metrics* (Herbert Ullrich, Tomáš Mlynář, Jan Drchal)

Query Optimization:

Source Authority Routing and Cross-Reference Validation:

-Metropolitansky, D., et al. (2025). Towards Effective Extraction and Evaluation of Factual Claims. Microsoft Research Special Projects. (Focus: Atomic claim isolation and context preservation).

-Kangur, U., Agrawal, K., Singh, Y., Sabir, A., & Sharma, R. (2025). MultiReflect: Multimodal Self-Reflective RAG-based Automated Fact-Checking. Proceedings of MAGMaR. (Focus: Retrieval-Augmented Generation and search query engineering).

-Wright, D., et al. (2025). Claim Extraction for Fact-Checking: Data, Models, and Automated Metrics. NLP Verification Frameworks. (Focus: Veracity classification and source mapping metrics).

Architectural Workflow:

[User Input — max ~500 words]



[1. Sentence Splitter]

- Split paragraph into individual sentences
- Library: `nltk.sent_tokenize`



[2. Atomic Claim Extractor] ← Fast Model (Ollama)

- Per sentence: is this a verifiable factual claim?
- Filters out opinions, questions, filler
- Output: list of clean factual claim strings



[3. NER / Keyphrase Extractor] ← spaCy (0 VRAM)

- Extract named entities per claim
(persons, dates, places, events)
- Build a Wikipedia-searchable query string



[4. Wikipedia Retriever] ← LangChain WikipediaRetriever

- Query Wikipedia per claim
- Fetch top 2 articles



[5. Semantic Chunker] ← sentence-transformers (CPU)

- Split articles into chunks
- Return top-3 most relevant chunks per claim



[6. Reasoning / Veracity Model] ← Larger Model (Ollama)

- Input: claim + top-3 chunks
- Output: Supported / Refuted / Not Enough Info



[7. Post Processor]

- Attach Wikipedia source URL to each result
- Aggregate all claim results into final response



[Response to Frontend]

Folder Structure:

AutoCitation/

```
├── Backend/
│   ├── main.py          ← FastAPI entry point
│   ├── pipeline/
│   │   ├── claim_extractor.py ← Step 1: claim extraction (fast model)
│   │   ├── ner.py         ← Step 3: spaCy NER + query builder
│   │   ├── retriever.py    ← Step 4: Wikipedia retrieval
│   │   ├── chunker.py     ← Step 5: semantic chunking
│   │   ├── verifier.py    ← Step 6: veracity reasoning (large model)
│   │   └── postprocessor.py ← Step 7: format + attach URLs
│   └── models/
│       └── ollama_client.py ← shared Ollama wrapper for both models
│   ├── config.py        ← model names, word limit, thresholds
│   └── requirements.txt
├── Frontend/
│   └── index.html       ← single page, textarea input + results table
├── Documents/
│   └── AutoCitation_DesignPlan_Ba_V0_1.pdf
└── readme.md
```

3.Success Criteria

A. Claim Extraction Precision => 85%

- **What it measures:** Out of all the statements your lightweight model (like Phi-4 Mini) claims are "facts," how many are actually verifiable facts versus opinions or filler text?
- **Formula:**

Precision = True Factual Claims Found\Total Claims Extracted by Model

B. Veracity Classification F1-Score => 80%

- **What it measures:** When your reasoning model (DeepSeek-R1) labels a claim as *Supported*, *Refuted*, or *Not Enough Info*, how often is it completely correct?
- **Why F1-Score?** In fact-checking, an F1-score is the industry standard because it balances **Precision** (avoiding false accusations of lying) and **Recall** (not missing actual lies). Academically, hitting an 80% F1-score on a custom local RAG pipeline is considered an excellent, production-ready benchmark.

C. Groundedness / Faithfulness Score => 90%

- **What it measures:** Does the AI strictly use the text inside the retrieved search snippet to verify the claim, or is it making things up using its own internal memory (hallucinating)?
- **The Rule:** If a search snippet does not contain the answer, the model *must* output "Not Enough Info." If it tries to guess, it fails this metric.

Metric Category	Success Threshold	Why this standard matters
VRAM Consumption	< 70% of total GPU VRAM	Ensures Ollama leaves enough VRAM for your operating system and display drivers to prevent freezing.
Inference Velocity	> 25 tokens / second	The model must process text fast enough that a user isn't waiting minutes for a single paragraph to check.
End-to-End Latency	< 8 seconds per sentence	Total time to extract a claim, hit the search API, read the snippet, and output the colored truth verdict.

